



**UNIVERSIDADE FEDERAL DO RURAL DO SEMI-ÁRIDO CENTRO
MULTIDISCIPLINAR DE PAU DOS FERROS DEPARTAMENTO DE
ENGENHARIAS E TECNOLOGIA PROGRAMAÇÃO CONCORRENTE E
DISTRIBUÍDA DOCENTE: ÍTALO AUGUSTO SOUZA DE ASSIS**

Allyson Bruno de Freitas Fernandes

PROGRAMAÇÃO CONCORRENTE E DISTRIBUÍDA

PAU DOS FERROS – RN 2026

QUESTÕES RESPONDIDAS

Lista de Exercícios - Porque computação paralela?

[QUESTÃO 01 - RESPONDIDA](#)

[QUESTÃO 02 - RESPONDIDA](#)

[QUESTÃO 03 - RESPONDIDA](#)

[QUESTÃO 04 - RESPONDIDA](#)

[QUESTÃO 05 - RESPONDIDA](#)

[QUESTÃO 06 - RESPONDIDA](#)

[QUESTÃO 07 - RESPONDIDA](#)

Lista de Exercícios - Hardware paralelo

[QUESTÃO 01 - RESPONDIDA](#)

[QUESTÃO 02 - RESPONDIDA](#)

[QUESTÃO 03 - RESPONDIDA](#)

[QUESTÃO 04 - RESPONDIDA](#)

[QUESTÃO 05 - RESPONDIDA](#)

[QUESTÃO 06 - RESPONDIDA](#)

Lista de Exercícios - Software paralelo

[QUESTÃO 01 - RESPONDIDA](#)

[QUESTÃO 02 - RESPONDIDA](#)

[QUESTÃO 03 - RESPONDIDA](#)

[QUESTÃO 04 - RESPONDIDA](#)

[QUESTÃO 05 - RESPONDIDA](#)

[QUESTÃO 06 - RESPONDIDA](#)

Lista de Exercícios - Porque computação paralela?

1. Suponha que precisamos calcular n valores e somá-los. Suponha que também tenhamos p núcleos e p seja muito menor que n . Então cada núcleo pode calcular uma soma parcial de aproximadamente valores n/p da seguinte maneira:

C/C++

```
minha_soma = 0; meu_pri_i = . . . ; meu_ult_i = . . . ;  
  
for (meu_i = meu_pri_i; meu_i < meu_ult_i; meu_i+=meu_desl) {  
    meu_x = Compute_prox_valor(. . .);  
  
    minha_soma += meu_x;  
}
```

Aqui o prefixo *meu_* indica que cada núcleo está usando suas próprias variáveis privadas e cada núcleo pode executar este bloco de código independentemente dos outros núcleos.

Supondo que cada chamada para *Compute_prox_valor* requer aproximadamente a mesma quantidade de trabalho, elabore fórmulas para calcular *meu_pri_i*, *meu_ult_i* e *meu_desl*. Lembre-se de que cada núcleo deve receber aproximadamente o mesmo número de elementos de computação no loop.

Dica: primeiro considere o caso em que n é divisível por p . A partir daí, elabore fórmulas para o caso em que n não é divisível por p .

Resposta:

Caso n divisível por p : cada núcleo recebe exatamente n/p iterações. As fórmulas são diretas:

```
meu_pri_i = meu_rank * (n / p)
meu_ult_i = meu_pri_i + (n / p)
meu_desl = 1
```

Caso geral (n não necessariamente divisível por p): divide-se n por p (divisão inteira) obtendo quociente e resto. Os núcleos com rank menor que o resto recebem uma iteração extra, de modo que a diferença entre núcleos seja de no máximo 1:

```
quociente = n / p
resto     = n % p
```

```
se meu_rank < resto:
```

```
    meu_pri_i = meu_rank * (quociente + 1)
    meu_ult_i = meu_pri_i + (quociente + 1)
```

```
senão:
```

```
    meu_pri_i = meu_rank * quociente + resto
    meu_ult_i = meu_pri_i + quociente
```

```
meu_desl = 1
```

Com essas fórmulas, cada núcleo recebe $\lceil n/p \rceil$ ou $\lfloor n/p \rfloor$ iterações, garantindo balanceamento de carga máximo.

2. **Suponha que precisamos calcular n valores e somá-los. Suponha também que tenhamos p núcleos e p seja muito menor que n . Então cada núcleo pode calcular uma soma parcial de aproximadamente n/p valores da seguinte maneira:**

C/C++

```
minha_soma = 0;
meu_pri_i = . . . ;
meu_ult_i = . . . ;

for (meu_i = meu_pri_i; meu_i < meu_ult_i; meu_i+=meu_desl) {
    meu_x = Compute_prox_valor(. . .);
    minha_soma += meu_x;
}
```

Aqui o prefixo *meu_* indica que cada núcleo está usando suas próprias variáveis privadas e cada núcleo pode executar este bloco de código independentemente dos outros núcleos.

Considerando que a chamada com $i = k$ requer $k + 1$ vezes mais trabalho que a chamada com $i = 0$ (se a chamada com $i = 0$ requer 2 milissegundos, a chamada com $i = 1$ requer 4, a chamada com $i = 2$ requer 6...), elabore fórmulas para calcular *meu_pri_i*, *meu_ult_i* e *meu_desl*. Comparado com a distribuição em blocos contíguos de iterações, sua fórmula deve reduzir a diferença do tempo de execução entre os núcleos.

Resposta:

Quando as iterações possuem custo crescente, uma distribuição em blocos contíguos concentra o trabalho pesado em alguns núcleos. A solução é usar distribuição cíclica (round-robin): cada núcleo processa iterações espaçadas de p posições. Assim, iterações leves e pesadas ficam intercaladas entre todos os núcleos.

*$meu_pri_i = meu_rank$
 $meu_ult_i = n$
 $meu_desl = p$*

Com a distribuição cíclica, o núcleo k processa os índices $k, k+p, k+2p, \dots$. O custo total de cada núcleo fica próximo à média, pois as iterações caras (índices altos) são distribuídas igualmente. Isso reduz significativamente o desbalanceamento de carga em comparação ao esquema de blocos contíguos.

3. Escreva um pseudocódigo para uma soma global estruturada em árvore. Suponha que o número de núcleos seja uma potência de dois.

Resposta:

Na soma em árvore, a cada rodada metade dos núcleos envia seu resultado parcial para o parceiro correspondente, que acumula. O número de rodadas é $\log_2(p)$.

JavaScript

```
// cada núcleo já tem minha_soma (sua soma parcial local)

passo = 1

enquanto passo < p faça
  se meu_rank % (2 * passo) == 0 então
    parceiro = meu_rank + passo
    se parceiro < p então
      // receber valor de nucleo[parceiro]
      minha_soma = minha_soma + valor
  senão se meu_rank % passo == 0 então
    enviar minha_soma para nucleo[meu_rank - passo]
  fim se
  passo = passo * 2
fim enquanto
```

```
// núcleo 0 contém a soma global
```

4. **Podemos usar os operadores bit a bit de C para implementar uma soma global estruturada em árvore. Implemente este algoritmo em pseudocódigo usando o operador OU EXCLUSIVO e o operador *shift* para esquerda.**

Resposta:

O operador XOR com uma máscara de bits identifica o parceiro de cada núcleo em cada nível da árvore. O operador AND verifica se o núcleo deve receber ou enviar:

JavaScript

```
bitmask = 1
enquanto bitmask < p faça
    parceiro = meu_rank XOR bitmask           // identifica o parceiro
    se meu_rank AND bitmask == 0 então        // este núcleo recebe
        receber valor de nucleo[parceiro]
        minha_soma = minha_soma + valor
    senão                                     // este núcleo envia
        enviar minha_soma para nucleo[parceiro]
    fim se
    bitmask = bitmask << 1                   // próximo nível da árvore
fim enquanto
// núcleo 0 contém a soma global
```

A operação XOR com a máscara garante que, em cada nível, cada núcleo se comunique com exatamente um parceiro diferente. O shift desloca a máscara um bit à esquerda a cada rodada, subindo um nível na árvore.

5. **Escreva um pseudocódigo para uma soma global estruturada em árvore. Considere que o número de núcleos pode não ser uma potência de dois.**

Resposta:

Quando p não é potência de 2, os núcleos excedentes enviam seus valores antecipadamente para os núcleos dentro da maior potência de 2 menor ou igual a p . Em seguida, executa-se a soma em árvore padrão entre os núcleos válidos:

JavaScript

```
pot2 = maior potência de 2 tal que pot2 <= p

// Fase 1: núcleos excedentes enviam para parceiros dentro de pot2
se meu_rank >= pot2 então
    enviar minha_soma para nucleo[meu_rank - pot2]
senão se (meu_rank + pot2) < p então
    receber valor de nucleo[meu_rank + pot2]
    minha_soma = minha_soma + valor
fim se

// Fase 2: soma em árvore padrão entre núcleos 0..pot2-1
se meu_rank < pot2 então
    passo = 1
    enquanto passo < pot2 faça
        se meu_rank % (2 * passo) == 0 então
            parceiro = meu_rank + passo
            se parceiro < pot2 então
                receber valor de nucleo[parceiro]
                minha_soma = minha_soma + valor
            fim se
        senão se meu_rank % (2 * passo) == passo então
            enviar minha_soma para nucleo[meu_rank - passo]
        fim se
        passo = passo * 2
    fim enquanto
fim se

// núcleo 0 contém a soma global
```

6. Elabore fórmulas para o número de recebimentos e adições que o núcleo 0 realiza usando

(a) uma soma global onde apenas o núcleo 0 realiza as adições;

Resposta:

Soma simples (só o núcleo 0 soma): o núcleo 0 recebe de cada um dos $p-1$ demais núcleos e realiza $p-1$ adições.

Recebimentos = $p - 1$ Adições = $p - 1$

(b) uma soma global estruturada em árvore.

Resposta:

Soma em árvore: o núcleo 0 participa de $\log_2(p)$ rodadas, recebendo e somando uma vez por rodada.

$$\text{Recebimentos} = \log_2(p) \quad \text{Adições} = \log_2(p)$$

p	Soma simples (recv)	Soma simples (add)	Soma em árvore (recv)	Soma em árvore (add)
2	1	1	1	1
4	3	3	2	2
8	7	7	3	3
16	15	15	4	4
32	31	31	5	5
64	63	63	6	6
128	127	127	7	7
256	255	255	8	8
512	511	511	9	9
1024	1023	1023	10	10

A vantagem da soma em árvore é evidente: para 1024 núcleos, o núcleo 0 faz apenas 10 operações em vez de 1023. Isso representa uma redução de mais de 100 vezes no trabalho do núcleo mestre.

Faça uma tabela mostrando os números de recebimentos e adições realizados pelo núcleo 0 quando as duas somas são usadas com 2, 4, 8, $\dots$, 1024 núcleos.

7. **Descreva um problema de pesquisa em sua área (Tecnologia de Informação, Engenharia de Software...) que se beneficiaria com o uso da computação paralela. Forneça um esboço de como o paralelismo seria usado. Você usaria paralelismo de tarefas ou de dados? Por quê?**

Resposta:

Área: Engenharia de Software / Inteligência Artificial

Problema: Treinamento de modelos de aprendizado de máquina para classificação de imagens médicas (como detecção de tumores em exames de ressonância magnética).

Esboço: o conjunto de treinamento pode conter milhões de imagens. Em vez de processá-las sequencialmente, divide-se o dataset entre os núcleos disponíveis. Cada núcleo processa seu

subconjunto de imagens, calcula os gradientes locais do modelo e, ao fim de cada iteração, os gradientes são agregados (via soma global em árvore) para atualizar os pesos do modelo de forma consistente.

Tipo de paralelismo: paralelismo de dados. Todos os núcleos executam a mesma operação (cálculo de gradiente) sobre partes diferentes dos dados. Isso é adequado pois o dataset é homogêneo e o volume de dados é o principal gargalo. O paralelismo de dados permite escalar o treinamento de semanas para horas ao aumentar o número de núcleos, sem alterar a lógica do algoritmo.

Lista de Exercícios - Hardware paralelo

1. Quando discutimos a adição de ponto flutuante, partimos da suposição simplificadora de que cada uma das unidades funcionais levava o mesmo tempo. Suponha que buscar (*fetch*) e armazenar (*store*) levem 2 nanossegundos cada e que as operações restantes levem 1 nanossegundo cada.

- (a) Quanto tempo leva uma adição de ponto flutuante com essas suposições?

Resposta:

A adição de ponto flutuante tem 7 estágios: fetch, comparar expoentes, alinhar mantissa, somar mantissas, normalizar, arredondar, store. Com fetch e store = 2 ns e os demais = 1 ns:

Tempo de uma adição:

$$2 + 1 + 1 + 1 + 1 + 1 + 2 = 9 \text{ ns}$$

- (b) Quanto tempo levará uma adição sem pipeline de 1.000 pares de *floats* com essas suposições?

Resposta:

Sem pipeline – 1.000 pares:

$$1.000 \times 9 \text{ ns} = 9.000 \text{ ns} = 9 \mu\text{s}$$

- (c) Quanto tempo levará uma adição em pipeline de 1.000 pares de *floats* com essas suposições?

Resposta:

O primeiro resultado sai após os 7 estágios iniciais (latência de enchimento). Em seguida, os 999 resultados restantes saem um por ciclo de 2 ns. Tempo total = $(6 + 1000) \times 2 \text{ ns} = 2.012 \text{ ns}$. Aceleração $\approx 9.000 / 2.012 \approx 4,5\times$.

Com pipeline – 1.000 pares: o gargalo é o estágio mais lento (fetch/store = 2 ns), portanto o ciclo do pipeline é 2 ns. Após encher o pipeline nos primeiros 7 estágios, cada resultado sai a cada 2 ns:

$$\text{Tempo total} \approx (7 + 999) \times 2 \text{ ns} = 1.006 \times 2 \text{ ns} = 2.012 \text{ ns} \approx 2 \mu\text{s}$$

$$\text{Aceleração} \approx 9.000 / 2.012 \approx 4,5\times$$

- (d) O tempo necessário para busca e armazenamento pode variar consideravelmente se os operandos/resultados forem armazenados em diferentes níveis da hierarquia de memória. Suponha que uma busca de um cache de nível 1 leve dois nanossegundos, enquanto uma busca de um cache de nível 2 leve cinco nanossegundos e uma busca da memória principal leve cinquenta nanossegundos. O que acontece com o pipeline quando há uma falha de

cache de nível 1 na busca de um dos operandos? O que acontece quando há uma falha de nível 2?

Resposta:

Impacto de falhas de cache:

Falha de L1 (busca em L2 = 5 ns): o estágio de fetch, normalmente 2 ns, passa a durar 5 ns. O pipeline precisa inserir ciclos de espera (pipeline stalls): os estágios seguintes ficam ociosos até o operando chegar. O throughput cai enquanto dura a falha.

Falha de L2 (busca na memória principal = 50 ns): impacto muito mais grave. O fetch dura 50 ns — 25 ciclos de stall. Durante esse tempo todo o pipeline para. Em cargas com alta taxa de falha de L2, o benefício do pipeline é quase completamente anulado

Quantificando os ciclos de stall:

O ciclo do pipeline é determinado pelo estágio mais lento (fetch = 2 ns). Falha de L1 -> busca em L2 (5 ns): Ciclos de stall = $(5 - 2) / 1 = 3$ ciclos de espera Os estágios seguintes ficam com 'bolha' por 3 ciclos. Impacto moderado no throughput. Falha de L2 -> busca na memória principal (50 ns): Ciclos de stall = $(50 - 2) / 1 = 48$ ciclos de espera O pipeline fica paralisado por 48 ciclos. Em cargas com alta taxa de falha de L2, o benefício do pipeline é quase completamente anulado.

2. Explique como uma fila implementada em hardware na CPU poderia ser usada para melhorar o desempenho de um *cache write-through*.

Resposta:

No cache write-through, toda escrita na cache é propagada imediatamente para a memória principal. Como a memória é muito mais lenta que a cache, a CPU ficaria bloqueada aguardando cada escrita completar antes de continuar.

Uma fila de escritas em hardware (write buffer) resolve esse problema: ao escrever, a CPU deposita o dado na fila e continua executando imediatamente. A fila, de forma assíncrona, envia as escritas pendentes para a memória principal em paralelo com a execução do programa.

Enquanto a fila não estiver cheia, a CPU nunca fica bloqueada por operações de escrita. O desempenho se aproxima do de um cache write-back, porém com a simplicidade de coerência do write-through.

3. Dado o exemplo a seguir envolvendo leituras de cache de um array bidimensional.

double A[MAX][MAX], x[MAX], y[MAX];

...

// Inicializa A e x; y = 0

...

**// Primeiro par de loops/ for (i = 0;
i < MAX ; i++)**

```
    for (j = 0; j < MAX; j++) y [i]
        += A[i][j]x[j];
    ...
// y = 0
...
// Segundo par de loops for (j =
0; j < MAX ; j++)
    for (i = 0; i < MAX; i++) y [i]
        += A[i][j]x[j];
```

- (a) Como o aumento do tamanho da matriz afetaria o desempenho dos dois pares de loops aninhados?

Resposta:

Efeito do aumento do tamanho da matriz:

Primeiro par (i externo, j interno): percorre A linha por linha — aproveita a localidade espacial. Com matrizes maiores, o desempenho se degrada pouco, pois cada linha carregada na cache é usada por completo antes de ser descartada.

Segundo par (j externo, i interno): percorre A coluna por coluna — viola a localidade espacial. Com matrizes maiores, cada acesso a $A[i][j]$ tende a trazer uma nova linha de cache que será usada apenas uma vez antes de ser substituída. O número de falhas cresce com MAX^2 , degradando o desempenho de forma muito mais acentuada.

- (b) Como o aumento do tamanho da cache afetaria o desempenho dos dois pares de loops aninhados?

Resposta:

Efeito do aumento da cache:

Ambos os pares se beneficiam. O segundo par beneficia-se mais: com cache suficientemente grande para conter toda a matriz, as falhas de acesso por coluna desaparecem, igualando o desempenho ao primeiro par.

- (c) Quantas falhas ocorrem nas leituras de A no primeiro par de loops aninhados?

Resposta:

Falhas no primeiro par (i externo, j interno):

A cada 4 elementos consecutivos acessados em uma linha, apenas 1 falha ocorre (a cache carrega 4 elementos por vez). Total de falhas $\approx MAX \times (MAX/4) = MAX^2/4$.

- (d) Quantas falhas ocorrem no segundo par?

Resposta:

Cada acesso a $A[i][j]$ salta MAX elementos na memória. A linha de cache carregada anteriormente provavelmente já foi substituída. Portanto, quase toda leitura gera falha:

Total de falhas $\approx MAX^2$ (uma por acesso, quando MAX é grande o suficiente para exceder a cache).

***Caso complementar:** quando MAX é pequeno o suficiente para a matriz inteira caber na cache, as linhas carregadas na primeira passagem permanecem disponíveis nas passagens seguintes. Nesse caso, o segundo par não teria mais falhas que o primeiro — ambos convergem para $MAX^2/4$. O resultado MAX^2 só vale quando a matriz excede a capacidade da cache.*

Suponha que uma linha de cache consiga armazenar quatro elementos de A .

4. Responda as questões abaixo. Explique o porquê de cada resposta.

- (a) A adição de cache e memória virtual a um sistema von Neumann altera sua designação como sistema SISD?

Resposta:

Não alteram a classificação SISD. São otimizações do subsistema de memória que melhoram o desempenho, mas o processador ainda executa uma única instrução operando sobre um único dado por vez. O fluxo de controle e de dados permanece único.

- (b) E quanto à adição de pipeline?

Resposta:

Não altera a classificação SISD. O pipeline sobrepõe estágios de execução de instruções distintas para aumentar o throughput, mas do ponto de vista lógico cada instrução ainda opera sobre um único fluxo de dados. O paralelismo é interno e não exposto como múltiplos fluxos independentes.

- (c) Multiple issue?

Resposta:

Pode aproximar o sistema de MIMD, mas muitos autores ainda classificam como SISD estendido. Com múltiplas unidades funcionais executando instruções distintas simultaneamente sobre dados distintos, há tecnicamente múltiplos fluxos, mas a perspectiva do programador continua sendo a de um único fluxo sequencial.

- (d) Hardware multithreading?

Resposta:

Aproxima o sistema da categoria MIMD. Com múltiplas threads intercaladas (fine-grained) ou executadas concorrentemente (SMT), há de fato múltiplos fluxos de instruções e dados ativos no mesmo núcleo. A classificação exata varia conforme a implementação.

5. Suponha que um programa deva executar 10^{12} instruções para resolver um problema específico. Suponha ainda que um sistema de processador único possa resolver o problema em 10^6 segundos (cerca de 11, 6 dias). Portanto, em média, o sistema de processador único executa 10^6 instruções por segundo. Agora suponha que o programa tenha sido pa-ralelizado para execução em um sistema de memória distribuída. Suponha também que o programa paralelo usa p processadores e, assim, cada processador executará $10^{12}/p$ instruções. Além disso, cada processador deverá enviar $10^9(p - 1)$ mensagens. Por fim, suponha que não haja *overhead* adicional na execução do programa paralelo. Ou seja, o programa será concluído depois que cada processador tiver executado todas as suas instruções e enviado todas as suas mensagens e não haverá atrasos devido a coisas como espera por mensagens.

- (a) Suponha que demore 10^{-9} segundos para enviar uma mensagem. Quanto tempo levará para o programa ser executado com 1000 processadores se cada processador for tão rápido quanto o processador único no qual o programa serial foi executado?

Resposta:

Latência de 10^{-9} s, $p = 1000$:

Tempo de computação = $(10^{12} / 1000) / 10^6 = 10^9 / 10^6 = 1.000$ s

Tempo de comunicação = $10^9 \times (1000 - 1) \times 10^{-9} = 999$ s

Tempo total $\approx 1.000 + 999 = 1.999$ s ≈ 33 minutos

Speedup $\approx 10^6 / 1.999 \approx 500 \times$

- (b) Suponha que demore 10^{-3} segundos para enviar uma mensagem. Quanto tempo levará para o programa rodar com 1000 processadores?

Resposta:

Latência de 10^{-3} s, $p = 1000$:

Tempo de computação = $(10^{12} / 1000) / 10^6 = 1.000$ s

Tempo de comunicação = $10^9 \times 999 \times 10^{-3} \approx 10^9$ s

Tempo total $\approx 10^9$ s

6. (a) Suponha que um sistema de memória compartilhada use coerência de cache (*sno-oping*) e caches *write-back*. Suponha também que o núcleo 0 tenha a variável x em seu cache e execute a atribuição $x = 5$. Finalmente, suponha que o núcleo 1 não tenha x em seu cache e, após a atualização do núcleo 0 para x , o núcleo 1 tente executar $y = x$. Qual valor será atribuído a y ? Por que?

Resposta:

Sistema com snooping cache (write-back): $y = 5$.

Quando o núcleo 0 executa $x = 5$, atualiza x em seu cache local e transmite essa informação pelo barramento compartilhado. O núcleo 1, monitorando o barramento (snooping), detecta que a

linha de cache com x foi modificada e invalida sua cópia (caso tivesse). Ao tentar ler x , o núcleo 1 busca o valor atualizado via barramento obtendo 5 e y recebe esse valor.

- (b) Suponha que o sistema de memória compartilhada da parte anterior utilize um protocolo baseado em diretório. Qual valor será atribuído a y ? Por que?

Resposta:

Sistema com protocolo baseado em diretório: $y = 5$.

O diretório mantém registro de quais núcleos possuem cópias de cada linha de cache. Quando o núcleo 0 escreve $x = 5$, notifica o diretório, que envia mensagens de invalidação para todos os núcleos com cópias de x . Quando o núcleo 1 solicita x , o diretório redireciona a requisição ao núcleo 0, que fornece o valor atualizado (5).

- (c) Como os problemas encontrados nas duas primeiras partes podem ser resolvidos?

Resposta:

Ambos os protocolos garantem $y = 5$ por meios diferentes:

Snooping: usa broadcast no barramento. Simples, mas escala mal com muitos núcleos (contention no barramento).

Diretório: usa comunicação ponto a ponto via diretório centralizado/distribuído. Mais complexo, mas escala bem para sistemas com muitos núcleos.

A solução para os problemas de coerência em cada caso é justamente a implementação desses protocolos sem eles, leituras obsoletas (stale reads) ocorreriam e y poderia receber um valor desatualizado de x .

Lista de Exercícios - Software paralelo

1. Sendo n o tamanho do problema e p o número de *threads* (ou processos):

- (a) Suponha que o tempo de execução de um programa sequencial seja dado por $T_{\text{sequencial}} = n^2$, onde as unidades do tempo de execução estão em microssegundos. Suponha que uma paralelização deste programa tenha tempo de execução $T_{\text{paralelo}} = n^2/p + \log_2(p)$. Escreva um programa que encontre as acelerações e eficiências deste programa para vários valores de n e p . Execute seu programa com $n = 10, 20, 40, \dots, 320$ e $p = 1, 2, 4, \dots, 128$.
- O que acontece com os *speedups* e eficiências à medida que p aumenta e n é mantido fixo?
 - O que acontece quando p é fixo e n é aumentado?

Resposta:

```
C/C++
#include <stdio.h>
#include <math.h>

int main() {
    int n_vals[] = {10, 20, 40, 80, 160, 320};
    int p_vals[] = {1, 2, 4, 8, 16, 32, 64, 128};
    int nn = 6, np = 8;

    printf("%-6s %-6s %-14s %-14s\n", "n", "p", "Speedup", "Eficiencia");

    for (int i = 0; i < nn; i++) {
        double n = n_vals[i];
        for (int j = 0; j < np; j++) {
            double p = p_vals[j];
            double Tseq = n * n;
            double Tpar = (n * n) / p + log2(p);
            double S = Tseq / Tpar;
            double E = S / p;
            printf("%-6.0f %-6.0f %-14.4f %-14.4f\n", n, p, S, E);
        }
    }
    return 0;
}
```

n	p	Speedup	Eficiência
10	1	1.0000	1.0000
10	2	1.9608	0.9804

10	4	3.7037	0.9259
10	8	6.4516	0.8065
10	16	9.7561	0.6098
10	32	12.3077	0.3846
10	64	13.2231	0.2066
10	128	12.8514	0.1004
20	1	1.0000	1.0000
20	2	1.9900	0.9950
20	4	3.9216	0.9804
20	8	7.5472	0.9434
20	16	13.7931	0.8621
20	32	22.8571	0.7143
20	64	32.6531	0.5102
20	128	39.5062	0.3086
40	1	1.0000	1.0000
40	2	1.9975	0.9988
40	4	3.9801	0.9950
40	8	7.8818	0.9852
40	16	15.3846	0.9615
40	32	29.0909	0.9091
40	64	51.6129	0.8065
40	128	82.0513	0.6410
80	1	1.0000	1.0000
80	2	1.9994	0.9997
80	4	3.9950	0.9988
80	8	7.9701	0.9963
80	16	15.8416	0.9901

80	32	31.2195	0.9756
80	64	60.3774	0.9434
80	128	112.2807	0.8772
160	1	1.0000	1.0000
160	2	1.9998	0.9999
160	4	3.9988	0.9997
160	8	7.9925	0.9991
160	16	15.9601	0.9975
160	32	31.8012	0.9938
160	64	63.0542	0.9852
160	128	123.6715	0.9662
320	1	1.0000	1.0000
320	2	2.0000	1.0000
320	4	3.9997	0.9999
320	8	7.9981	0.9998
320	16	15.9900	0.9994
320	32	31.9501	0.9984
320	64	63.7609	0.9963
320	128	126.8897	0.9913

*Tabela montada com os dados reais da execução do seu programa. Verde = eficiência $\geq 0,90$
| Amarelo = 0,60–0,89 | Vermelho $< 0,60$.*

(b) Suponha que $T_{paralelo} = T_{sequencial}/p + T_{overhead}$. Suponha também que mantemos p fixo e aumentamos o tamanho do problema.

- Mostre que, se $T_{overhead}$ crescer mais lentamente que $T_{sequencial}$, a eficiência paralela aumentará à medida que aumentarmos o tamanho do problema.
- Mostre que se, por outro lado, $T_{overhead}$ crescer mais rápido que $T_{sequencial}$, a eficiência paralela diminuirá à medida que aumentamos o tamanho do problema.

Resposta:

A eficiência é:

$$E = T_{\text{sequencial}} / (p \times T_{\text{paralelo}}) = T_{\text{sequencial}} / (T_{\text{sequencial}} + p \times T_{\text{overhead}})$$

- Se T_{overhead} cresce mais lentamente que $T_{\text{sequencial}}$: ao aumentar o tamanho do problema, $T_{\text{sequencial}}$ cresce mais rápido que $p \times T_{\text{overhead}}$. O denominador é dominado por $T_{\text{sequencial}}$, e $E \rightarrow 1$. A eficiência aumenta.
- Se T_{overhead} cresce mais rapidamente que $T_{\text{sequencial}}$: $p \times T_{\text{overhead}}$ passa a dominar o denominador. Como $T_{\text{sequencial}}$ cresce mais devagar, a razão $T_{\text{sequencial}} / (T_{\text{sequencial}} + p \times T_{\text{overhead}}) \rightarrow 0$. A eficiência diminui.

2. Às vezes, diz-se que um programa paralelo que obtém um *speedup* maior que p (o número de processos ou *threads*) tem *speedup* superlinear. No entanto, muitos autores não consideram os programas que superam as "limitações de recursos" como tendo aceleração superlinear. Por exemplo, um programa que deve usar armazenamento secundário para seus dados quando é executado em um sistema de processador único pode ser capaz de acomodar todos os seus dados na memória principal quando executado em um grande sistema de memória distribuída. Dê outro exemplo de como um programa pode superar uma limitação de recursos e obter *speedups* maiores que p .

Resposta:

Um exemplo clássico além do armazenamento secundário é a busca com poda em algoritmos de otimização combinatória (como branch and bound).

Considere a busca pela solução ótima em uma árvore de decisão. No caso serial, o algoritmo percorre ramos que só são descartados depois de encontrar um limitante melhor. Com múltiplos processos explorando diferentes partes da árvore em paralelo, um processo pode encontrar rapidamente uma solução de boa qualidade e propagar esse limitante para os demais. Os outros processos, ao receberem esse limitante, passam a podar ramos que o programa serial ainda exploraria.

O resultado é que os processos coletivamente exploram menos nós do que o programa serial, levando a um $\text{speedup} > p$. Esse efeito não é artificial: surge da natureza do algoritmo de busca e da interação entre os processos durante a exploração paralela do espaço de soluções.

3. Sendo n o tamanho do problema e p o número de *threads* (ou processos), suponha $T_{\text{serial}} = n$ e $T_{\text{parallel}} = n/p + \log_2(p)$, onde os tempos estão em microssegundos. Se aumentarmos p por um fator de k , encontre uma fórmula para quanto precisaremos aumentar n para manter a eficiência constante.

A eficiência é:

$$E = T_{\text{serial}} / (p \times T_{\text{parallel}}) = n / (n + p \times \log_2(p))$$

Para manter E constante ao multiplicar p por k (novo $p' = k \cdot p$), igualamos as eficiências:

$$n / (n + p \cdot \log_2(p)) = n' / (n' + k \cdot p \cdot \log_2(k \cdot p))$$

Isolando n' (novo tamanho necessário):

$$n' = n \times k \times \log_2(k \cdot p) / \log_2(p)$$

(a) Quanto devemos aumentar n se dobrarmos o número de processos de 8 para 16?

Resposta:

Dobrando de $p = 8$ para $p = 16$ ($k = 2$):

$$n' = n \times 2 \times \log_2(16) / \log_2(8) = n \times 2 \times 4/3 = 8n/3 \approx 2,67n$$

Devemos aumentar n em aproximadamente 2,67 vezes para manter a eficiência constante.

(b) O programa paralelo é escalável?

Resposta:

*O programa **não é fortemente escalável**, pois ao fixar n e aumentar p o overhead $\log_2(p)$ cresce e a eficiência cai.*

*O programa **é fracamente escalável**, pois é possível manter E constante aumentando n junto com p . Porém há uma ressalva importante: n precisa crescer **mais que linearmente** com p , o que é uma limitação prática relevante:*

$$\text{Fórmula: } n' = n \times k \times \log_2(k \cdot p) / \log_2(p)$$

$$p = 8 \rightarrow 16 \text{ (} k=2 \text{): } n' = n \times 2 \times \log_2(16)/\log_2(8) = n \times 8/3 \approx 2,67n$$

$$p = 8 \rightarrow 32 \text{ (} k=4 \text{): } n' = n \times 4 \times \log_2(32)/\log_2(8) = n \times 20/3 \approx 6,67n$$

$$p = 8 \rightarrow 64 \text{ (} k=8 \text{): } n' = n \times 8 \times \log_2(64)/\log_2(8) = n \times 16 = 16n$$

Definições formais:

Forte escalabilidade: manter E constante aumentando apenas p , com n fixo $\rightarrow$ não atende.

Fraca escalabilidade: manter E constante aumentando n proporcionalmente ao aumento de $p \rightarrow$ atende, mas n cresce desproporcionalmente, o que em muitos cenários práticos pode ser inviável

4. Um programa que obtém *speedup* linear é fortemente escalável? Explique sua resposta.

Resposta:

*Não necessariamente. Forte escalabilidade exige que a eficiência $E = S/p$ seja mantida próxima de 1 ao aumentar p com o tamanho do problema fixo. Um programa com *speedup* linear ($S = p$) teria $E = 1$, o que seria ideal.*

*No entanto, na prática, *speedup* linear significa que $S = p$ para um determinado intervalo de n e p . Se o overhead de comunicação e sincronização cresce com p ainda que lentamente em algum ponto a eficiência começará a cair abaixo de 1 ao aumentar p com n fixo.*

Portanto, speedup linear é condição necessária mas não suficiente para forte escalabilidade. Na prática, qualquer overhead mesmo mínimo fará a eficiência cair ao aumentar p com n fixo, tornando a forte escalabilidade ideal inatingível na maioria dos sistemas reais.

5. **Bob tem um programa que deseja cronometrar com dois conjuntos de dados, *input_data1* e *input_data2*. Para ter uma ideia do que esperar antes de adicionar funções de tempo-rização ao código de seu interesse, ele executa o programa com os dois conjuntos de dados e o comando shell Unix *time*:**

```
$ time ./bobs_prog < input_data1 real
0m0.001 s
user 0m0.001 s
sys 0m0.000 s
```

```
$ time ./bobs_prog < input_data2 real
1m1.234 s
user 1m0.001 s
sys 0m0.111 s
```

A função de temporização que Bob está usando tem resolução de milissegundos.

- (a) Bob deveria usá-la para cronometrar seu programa com o primeiro conjunto de dados? Por que?

Resposta:

Primeiro conjunto (tempo real ≈ 1 ms): Não. Quando o tempo de execução é da mesma ordem de grandeza da resolução do instrumento (1 ms), o erro de medição pode ser de até 100% ou mais. O resultado seria impreciso e não confiável. Para medir esse caso adequadamente, seria necessário usar um instrumento de maior resolução (microsegundos) ou executar o programa milhares de vezes e medir o tempo total, dividindo pelo número de execuções.

- (b) E quanto ao segundo conjunto de dados? Por que?

Resposta:

Segundo conjunto (tempo real ≈ 61.234 ms): Sim, a função é plenamente adequada. O tempo de execução é mais de 61.000 vezes maior que a resolução do instrumento (1 ms), portanto o erro relativo de medição é inferior a 0,002%. Além disso, a diferença entre tempo real (61,234 s) e tempo de usuário (60,001 s) indica aproximadamente 1,2 s em overhead de sistema ($\text{sys} = 0,111 \text{ s} + \text{tempo de espera}$), o que pode ser relevante de analisar e a função capturará adequadamente.

6. Escreva um pseudocódigo para a soma global estruturada em árvore para somar vetores.

- (a) Primeiro considere como isso pode ser feito em um ambiente de memória comparti-lhada.
- Quais variáveis são compartilhadas e quais são privadas?

Resposta:

Cada thread calcula a soma parcial do seu segmento do vetor e armazena em um array compartilhado. Em seguida, a soma em árvore combina os resultados:

JavaScript

```
// Variáveis COMPARTILHADAS: vetor v[n], soma_parcial[p]
```

```

// Variáveis PRIVADAS:      meu_rank, meu_ini, meu_fim,
//                          soma_local, passo, parceiro, i

// Fase 1: cada thread soma seu segmento
soma_local = 0
para i de meu_ini até meu_fim:
    soma_local += v[i]
soma_parcial[meu_rank] = soma_local
barreira()    // espera todas as threads terminarem
// Fase 2: soma em árvore sobre soma_parcial[]
passo = 1
enquanto passo < p faça:
    se meu_rank % (2*passo) == 0 então:
        parceiro = meu_rank + passo
        se parceiro < p então:
            soma_parcial[meu_rank] += soma_parcial[parceiro]
    barreira()    // sincroniza cada nível da árvore
    passo = passo * 2
fim enquanto
// soma_parcial[0] contém a soma global do vetor

```

Variáveis compartilhadas: v[] (vetor original), soma_parcial[] (somadas parciais de cada thread).

Variáveis privadas: meu_rank, meu_ini, meu_fim (delimitam o segmento de cada thread), soma_local, passo, parceiro, i (auxiliares de controle).

(b) Depois considere como isso pode ser feito em um ambiente de memória distribuída.

Resposta:

Em memória distribuída não há variáveis compartilhadas. Cada processo possui apenas seu segmento local do vetor e se comunica via troca explícita de mensagens:

JavaScript

```

// Cada processo possui seu segmento local de v[]

// Fase 1: cada processo soma seu segmento local
minha_soma = 0
para cada elemento e no segmento local:
    minha_soma += e

// Fase 2: soma em árvore via troca de mensagens
passo = 1

```



```
enquanto passo < p faça:
    se meu_rank % (2*passo) == 0 então:
        parceiro = meu_rank + passo
        se parceiro < p então:
            receber valor_parceiro de processo[parceiro]
            minha_soma += valor_parceiro
        fim se
    senão se meu_rank % (2*passo) == passo então:
        enviar minha_soma para processo[meu_rank - passo]
    fim se
    passo = passo * 2
fim enquanto

// processo 0 contém a soma global do vetor

// Em memória distribuída, toda comunicação é explícita (send/receive).
// Não há barreiras globais entre os níveis, pois o sincronismo é
// garantido pelo próprio bloqueio nas operações de recebimento.
```


